

Alphabet:

- a. Upper (A-Z) and lower case letters (a-z) of the English alphabet
[A-Za-z]
- b. Decimal digits (0-9)
- c. Underline character ('_')

Lexic:

- a. Special characters, representing:

Operators:

Arithmetic: + (addition), - (subtraction), * (multiplication), ^ (power), / (division), % (mod)

Relational: == (equality), < (smaller), > (greater), <= (smaller or equal), >= (greater or equal), != (inequality)

Logical: & (and), | (or), ! (not)

Assignment: =

Index: []

Separators:

{ } () ; , <space> <newline>

Reserved words:

PROGRAM, FUNCTION, METHOD, read, write, if, else, for, do, while, break, int, char, string, list, bool, true, false return, EOP

- b. Identifiers:

A sequence of letters, underscores and digits; the first character is a letter; the rule is:

identifier = letter | letter{letter | underscore | digit}

letter = "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"

underscore = "_"

digit = "0" | "1" | ... | "9"

- c. Constants:

< TODO >

Syntax:

< TODO >

LAB1a

p1: max of 3 numbers

```
PROGRAM p1{
    int a, b, c, max;
    read(a, b, c);
    if(a>b & b>c){ max=a; }
    else{ if(b>c & b>a) { max=b; }
          else{ max=c; }
    }
    write(max);
    EOP;
}
```

p2: check if given number is prime

```
PROGRAM p2{
    int n, iterator;
    bool isPrime;    isPrime = true;
    if(n%2 == 0){ isPrime = false; write("Given number is not a
prime"); EOP;}
    for(iterator = 3; iterator*iterator <= n; iterator = iterator + 2){
if(a%iterator == 0){ write("Given number is not a prime"); EOP;}}
    write("Given number is a prime");
    EOP;
}
```

p3: compute the min/max of `n` given numbers

```
PROGRAM p3{
    int n, min, max;
    read(n);
    if(n<1){ write("Bad n given"); EOP;}
    int currentNumber;
    read(currentNumber); n = n-1;
    min = currentNumber; max = currentNumber;
    while(n!=0){ read(currentNumber); if(currentNumber>max){ max =
currentNumber; } if(currentNumber<min){ min = currentNumber;} n = n-1;}
    write("Min and max are ", min, "and ", max);
    EOP;
}
```

plerror:

```
PROGRAM plerror{
    inT a, b, c, max; <- error here since token "inT" does not exist
    read(a, b, c);
    if(a>b & b>c){ max=a; }
    else{ if(b>c && b>a) { max=b; } <- error here since token '&&' does
not exist
        else{ max=c; }
    }
    write(max);
    EOP;
}
```